

Lab 01. Building an Product Management Application using LINQ and WPF

1. Introduction

Imagine you're an employee of a store named **ProductStore**. Your manager has asked you to develop a WPF application for product management. The application has to support adding, viewing, modifying, and removing products—a standardized usage action verbs better known as Create, Read, Update, Delete (CRUD).

This lab explores creating an application using WPF with .NET Core, and C#. An "in-memory database" will be created to persist the car's data, so a collection is called List will be used for reading and managing product data.

2. Lab Objectives

In this lab, you will:

- Use the Visual Studio.NET to create WPF application and Class Library (.dll) project.
- Use the Visual Studio.NET to create Windows Forms and Class Library (.dll) project.
- Create a List of persisting products using LINQ to Object to find items.
- Apply passing data in WPF application
- Apply Repository pattern in a project.
- Add CRUD action methods to WPF application.
- Run the project and test the WPF application actions.

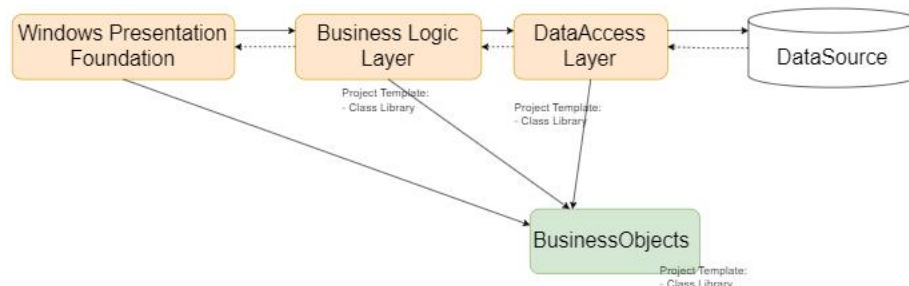
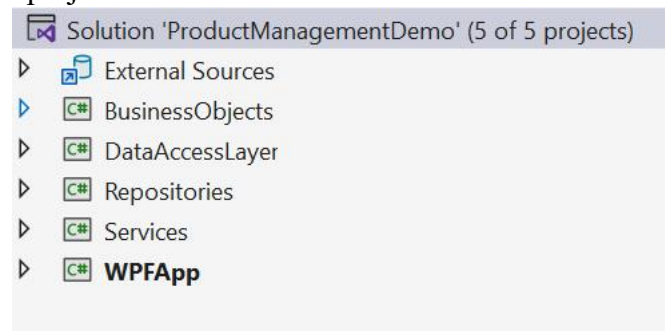
Activity 01: Build a solution by Visual Studio.NET

Create a Blank Solution named **ProductManagementDemo** then add new a **Class Library** project named **BusinessObjects**, **DataAccessObjects**, **Repositories**, **Services** and a WPF project named **ProductManagement**

Step 01. Create a Blank solution.

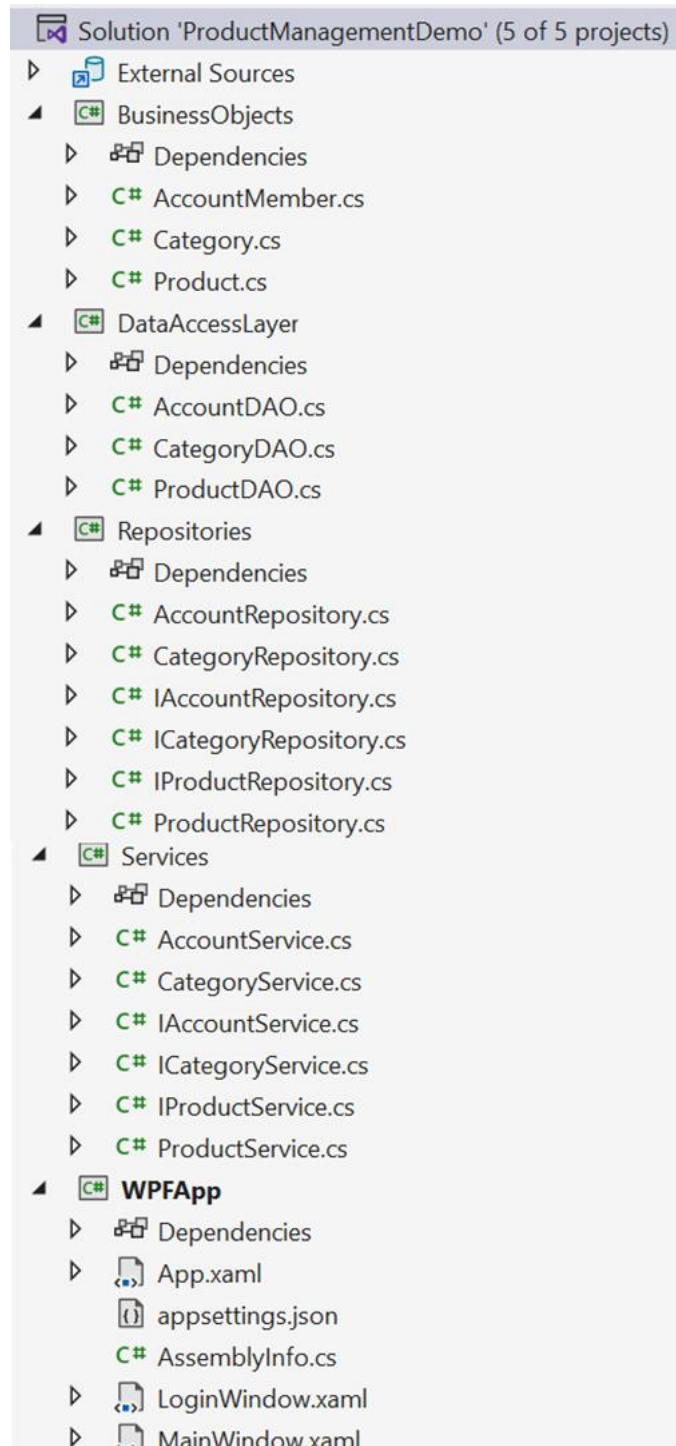
Step 02. Create 4 **Class Library** projects.

Step 03. Create a WPF project.

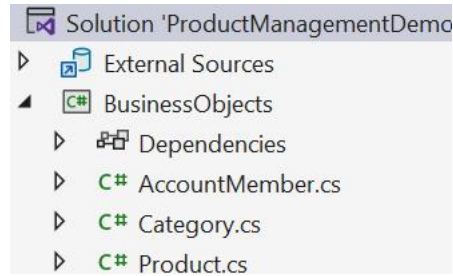


Note:

- **Data Source** in this case is the “**In memory Database**” (using List collection)
- **Services Project** – This project represents a layer or component responsible for implementing the business logic of an application.
- **Repository Project** – This project provides an abstraction layer between the application’s business logic and the underlying data source.
- **Data Access Layer Project** – This project used to abstract and encapsulate the logic for accessing data from a data source, such as a database.



Activity 02: Write codes for the BusinessObjects project



Step 01. On the **BusinessObjects** project, add a class named **Product.cs** and write codes as follows:

```

1  using System.Collections.Generic;
2
3  namespace BusinessObjects
4  {
5      25 references
6      public partial class Category
7      {
8          0 references
9          public Category()
10         {
11             Products = new HashSet<Product>();
12         }
13
14         8 references
15         public Category(int catID, string catName)
16         {
17             this.CategoryId = catID;
18             this.CategoryName = catName;
19         }
20
21         1 reference
22         public int CategoryId { get; set; }
23         1 reference
24         public string CategoryName { get; set; }
25
26         1 reference
27         public virtual ICollection<Product> Products { get; set; }
28     }
29 }

```

Step 02. On the **BusinessObjects** project, add a class named **Category.cs** and write codes as follows:

```

2  namespace BusinessObjects
3  {
4      46 references
5      public partial class Product
6      {
7          6 references
8          public Product(int id,string name,int catId,
9              short unitInStock,decimal price) {
10              this.ProductId= id;
11              this.ProductName = name;
12              this.CategoryId= catId;
13              this.UnitsInStock = unitInStock;
14              this.UnitPrice = price;
15          }
16          12 references
17          public int ProductId { get; set; }
18          7 references
19          public string ProductName { get; set; }
20          7 references
21          public int? CategoryId { get; set; }
22          7 references
23          public short? UnitsInStock { get; set; }
24          7 references
25          public decimal? UnitPrice { get; set; }
26          0 references
27          public virtual Category Category { get; set; }
28      }
29  }

```

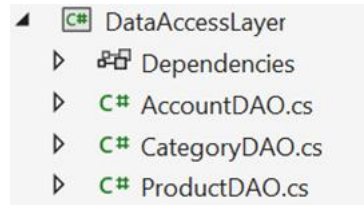
Step 02. On the **BusinessObjects** project, add a class named **Category.cs** and write codes as follows:

```

2  namespace BusinessObjects
3  {
4      8 references
5      public partial class AccountMember
6      {
7          1 reference
8          public string MemberId { get; set; } = null!;
9          2 references
10         public string MemberPassword { get; set; } = null!;
11         0 references
12         public string FullName { get; set; } = null!;
13         0 references
14         public string? EmailAddress { get; set; }
15         2 references
16         public int? MemberRole { get; set; }
17     }
18 }

```


Activity 03: Write codes for the DataAccessLayer project



Step 01. On the **DataAccessLayer** project, add a class named **CategoryDAO.cs** and write codes as follows:

```

2      using System.Collections.Generic;
3      using BusinessObjects;
4
5      namespace DataAccessLayer
6      {
7          1 reference
8          public class CategoryDAO
9          {
10             1 reference
11             public static List<Category> GetCategories()
12             {
13                 Category beverages = new Category(1, "Beverages");
14                 Category condiments = new Category(2, "Condiments");
15                 Category confections = new Category(3, "Confections");
16                 Category dairy = new Category(4, "Dairy Products");
17                 Category grains = new Category(5, "Grains/Cereals");
18                 Category meat = new Category(6, "Meat/Poultry");
19                 Category produce = new Category(7, "Produce");
20                 Category seafood = new Category(8, "Seafood");
21
22                 var listCategories = new List<Category>();
23                 try
24                 {
25                     listCategories.Add(beverages);
26                     listCategories.Add(condiments);
27                     listCategories.Add(confections);
28                     listCategories.Add(dairy);
29                     listCategories.Add(grains);
30                     listCategories.Add(meat);
31                     listCategories.Add(produce);
32                     listCategories.Add(seafood);
33                 }
34                 catch (Exception e)
35                 {
36                     throw new Exception(e.Message);
37                 }
38                 return listCategories;
39             }
40         }
41     }

```

Step 02. On the **DataAccessLayer** project, add a class named **ProductDAO.cs** and write codes as follows:

The details of functions in ProductDAO.cs:

```

2  using System.Collections.Generic;
3  using System.Linq;
4  using BusinessObjects;
5
6  namespace DataAccessLayer
7  {
8      3 references
9      public class ProductDAO
10     {
11         private static List<Product> listProducts;
12
13         1 reference
14         public ProductDAO()
15         {
16             Product chai = new Product(1, "Chai", 3, 12, 18);
17             Product chang = new Product(2, "Chang", 1, 23, 19);
18             Product aniseed = new Product(3, "Aniseed Syrup", 2, 23, 10);
19             listProducts = new List<Product> { chai, chang, aniseed };
20             //Product chef = new Product(4, "Chef Anton's Cajun Seasoning", 2, 34, 22);
21             //Product chefMix = new Product(5, "Chef Anton's Gumbo Mix", 2, 45, 34);
22             //Product grandma = new Product(6, "Grandma's Boysenberry Spread", 2, 21, 25);
23             //Product uncle = new Product(7, "Uncle Bob's Organic Dried Pears", 7, 22, 30);
24             //Product northwoods = new Product(8, "Northwoods Cranberry Sauce", 2, 10, 40);
25             //Product mishi = new Product(9, "Mishi Kobe Niku", 6, 12, 97);
26             //Product ikura = new Product(10, "Ikura", 8, 13, 32);
27             // listProducts = new List<Product> { chai, chang, aniseed, chef, chefMix, grandma
28         }
29
30         1 reference
31         public List<Product> GetProducts()
32         {
33             return listProducts;
34         }
35
36         public static List<Product> GetProducts()
37         {
38             var listProducts = new List<Product>();
39             try
40             {
41                 using var db = new MyStoreContext();
42                 listProducts = db.Products.ToList();
43             }
44             catch (Exception e) { }
45
46             return listProducts;
47         }
48     }

```

```

32  public void SaveProduct(Product p)
33  {
34      listProducts.Add(p);
35  }
36
37  1 reference
38  public void UpdateProduct(Product product)
39  {
40      foreach (Product p in listProducts.ToList())
41      {
42          if (p.ProductId == product.ProductId)
43          {
44              p.ProductId = product.ProductId;
45              p.ProductName = product.ProductName;
46              p.UnitPrice = product.UnitPrice;
47              p.UnitsInStock = product.UnitsInStock;
48              p.CategoryId = product.CategoryId;
49          }
50      }
51  }
52
53
54  public void DeleteProduct(Product product)
55  {
56      foreach (Product p in listProducts.ToList())
57      {
58          if (p.ProductId == product.ProductId)
59          {
60              listProducts.Remove(p);
61          }
62      }
63
64  1 reference
65  public Product GetProductById(int id)
66  {
67      foreach (Product p in listProducts.ToList())
68      {
69          if (p.ProductId == id)
70          {
71              return p;
72          }
73      }
74      return null;
75  }
76

```

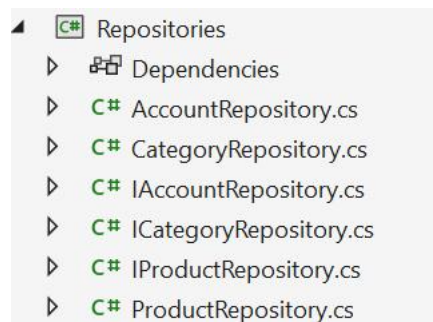

Step 03. Write codes for **AccountDAO.cs** as follows:

```

1  using BusinessObjects;
2
3  namespace DataAccessLayer
4  {
5      1 reference
6      public class AccountDAO
7      {
8          1 reference
9          public static AccountMember GetAccountById(string accountID)
10         {
11             AccountMember accountMember = new AccountMember();
12             if (accountID.Equals("PS0001")) // just for demonstration
13             {
14                 accountMember.MemberId = accountID;
15                 accountMember.MemberPassword = "@1";
16                 accountMember.MemberRole = 1;
17             }
18             return accountMember;
19         }
20     }
21 }

```

Activity 04: Write codes for the Repositories project



Step 01. On the **Repositories** project, add an interface named **ICategoryRepository.cs** and write codes as follows:

```

1  using System.Collections.Generic;
2  using BusinessObjects;
3
4  namespace Repositories
5  {
6      2 references
7      public interface ICategoryRepository
8      {
9          2 references
10         List<Category> GetCategories();
11     }
12 }

```

Step 02. On the **Repositories** project, add an interface named **IProductRepository.cs** and write codes as follows:

```

1  using System.Collections.Generic;
2  using BusinessObjects;
3
4  namespace Repositories
5  {
6      2 references
7      public interface IProductRepository
8      {
9          2 references
10         void SaveProduct(Product p);
11         2 references
12         void DeleteProduct(Product p);
13         2 references
14         void UpdateProduct(Product p);
15         2 references
16         List<Product> GetProducts();
17         2 references
18         Product GetProductById(int id);
19     }
20 }

```

Step 03. On the **Repositories** project, add an interface named **IAccountRepository.cs** and write codes as follows:

```

1  using BusinessObjects;
2
3  namespace Repositories
4  {
5      2 references
6      public interface IAccountRepository
7      {
8          2 references
9          AccountMember GetAccountById(string accountID);
10     }
11 }

```

Step 04. Write codes for class **CategoryRepository.cs** as follows:

```

1  using System.Collections.Generic;
2  using BusinessObjects;
3  using DataAccessLayer;
4
5  namespace Repositories
6  {
7      1 reference
8      public class CategoryRepository : ICategoryRepository
9      {
10         2 references
11         public List<Category> GetCategories() => CategoryDAO.GetCategories();
12     }
13 }

```

Step 05. Write codes for class **ProductRepository.cs** as follows:

```

1  using BusinessObjects;
2  using System.Collections.Generic;
3  using DataAccessLayer;
4
5  namespace Repositories
6  {
7      1 reference
8      public class ProductRepository : IProductRepository
9      {
10         2 references
11         public void DeleteProduct(Product p) => ProductDAO.DeleteProduct(p);
12         2 references
13         public void SaveProduct(Product p) => ProductDAO.SaveProduct(p);
14         2 references
15         public void UpdateProduct(Product p) => ProductDAO.UpdateProduct(p);
16         2 references
17         public List<Product> GetProducts() => ProductDAO.GetProducts();
18         2 references
19         public Product GetProductById(int id) => ProductDAO.GetProductById(id);
20     }
21 }

```

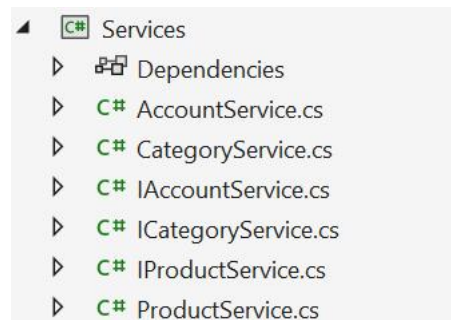
Step 06. Write codes for class **AccountRepository.cs** as follows:

```

1  using BusinessObjects;
2  using DataAccessLayer;
3
4  namespace Repositories
5  {
6      1 reference
7      public class AccountRepository : IAccountRepository
8      {
9          2 references
10         public AccountMember GetAccountById(string accountID)
11             => AccountDAO.GetAccountById(accountID);
12     }

```

Activity 05: Write codes for the Services project



Step 01. On the **Services** project, add an interface named **ICategoryService.cs** and write codes as follows:

```

1  using BusinessObjects;
2  using System.Collections.Generic;
3  namespace Services
4  {
5      2 references
6      public interface ICategoryService
7      {
8          2 references
9          List<Category> GetCategories();

```

Step 02. On the **Services** project, add an interface named **IProductService.cs** and write codes as follows:

```

1  using BusinessObjects;
2  using System.Collections.Generic;
3  namespace Services
4  {
5      public interface IProductService
6      {
7          void SaveProduct(Product p);
8          void DeleteProduct(Product p);
9          void UpdateProduct(Product p);
10         List<Product> GetProducts();
11         Product GetProductById(int id);
12     }
13 }

```

Step 03. On the **Services** project, add an interface named **IAccountService.cs** and write codes as follows:

```

1  using BusinessObjects;
2
3  namespace Services
4  {
5      public interface IAccountService
6      {
7          AccountMember GetAccountById(string accountID);
8      }
9  }

```

Step 04. Write codes for class **CategoryService.cs** as follows:


```

4  | using System.Collections.Generic;
5  | namespace Services
6  | {
7  |     2 references
8  |     public class CategoryService : ICategoryService
9  |     {
10 |         private readonly ICategoryRepository iCategoryRepository;
11 |
12 |         1 reference
13 |         public CategoryService()
14 |         {
15 |             iCategoryRepository = new CategoryRepository();
16 |
17 |         2 references
18 |         public List<Category> GetCategories()
19 |         {
20 |             return iCategoryRepository.GetCategories();
21 |         }
22 |     }
23 | }

```

Step 05. Write codes for class **ProductService.cs** as follows:

```

1  using BusinessObjects;
2  using Repositories;
3  using System.Collections.Generic;
4
5  namespace Services
6  {
7      2 references
8      public class ProductService : IProductService
9      {
10
11          1 reference
12          public ProductService()
13          {
14              iProductRepository = new ProductRepository();
15
16          2 references
17          public void DeleteProduct(Product p)
18          {
19              iProductRepository.DeleteProduct(p);
20
21          2 references
22          public Product GetProductById(int id)
23          {
24              return iProductRepository.GetProductById(id);
25
26          2 references
27          public List<Product> GetProducts()
28          {
29              return iProductRepository.GetProducts();
30
31          2 references
32          public void SaveProduct(Product p)
33          {
34              iProductRepository.SaveProduct(p);
35
36          2 references
37          public void UpdateProduct(Product p)
38          {
39              iProductRepository.UpdateProduct(p);
40          }
41      }
42  }

```

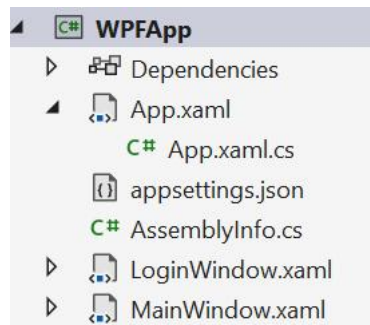
Step 06. Write codes for class **AccountService.cs** as follows:

```

1  ▼ using BusinessObjects;
2  [ using Repositories;
3
4  ▼ namespace Services
5  {
6      2 references
7      ▼ public class AccountService : IAccountService
8      {
9          1 reference
10         ▼ public AccountService() {
11             iAccountRepository = new AccountRepository();
12         }
13         2 references
14         ▼ public AccountMember GetAccountById(string accountID)
15         {
16             return iAccountRepository.GetAccountById(accountID);
17         }
18     }
19 }

```

Activity 04: Design UI and write codes for WPF project



Step 01. On the **WPF** project, design UI as follows:

Product Management

Product Management

Product ID

Product Name

Price

Units In Stock

Category

ProductId	ProductName	CategoryId	UnitsInStock	UnitPrice	Category
1	Chai	3	12	18.0000	
2	Chang	1	23	19.0000	
3	Aniseed Syrup	2	23	10.0000	
4	Chef Anton's Cajun Seasoning	2	34	22.0000	
5	Chef Anton's Gumbo Mix	2	45	21.3500	
6	Grandma's Boysenberry Spread	2	21	25.0000	
7	Uncle Bob's Organic Dried Pears	7	22	30.0000	
8	Northwoods Cranberry Sauce	2	10	40.0000	
9	Mishi Kobe Niku	6	12	97.0000	
10	Ikura	8	13	31.0000	

Create Update Delete Close

- XAML code for MainWindow.xaml

```
<Window x:Class="WPFApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WPFApp"
        mc:Ignorable="d"
        Loaded="Window_Loaded"
        WindowStartupLocation="CenterScreen"
        Title="Product Management" Height="670" Width="710">
    <Grid>
        <Grid>
            <Grid.Background>
                <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                    <GradientStop Color="#0099FF" Offset="0"/>
                    <GradientStop Color="#FF347BDA" Offset="0.794"/>
                    <GradientStop Color="#FF60B1E7" Offset="1"/>
                    <GradientStop Color="#FF596FDD" Offset="0.309"/>
                    <GradientStop Color="#FF472FDE" Offset="0.484"/>
                </LinearGradientBrush>
            </Grid.Background>
            <Grid.RowDefinitions>
                <RowDefinition Height="60"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
                <RowDefinition Height="30"/>
            </Grid.RowDefinitions>

            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="119.415"/>
                <ColumnDefinition Width="30.585"/>
                <ColumnDefinition Width="47*"/>
                <ColumnDefinition Width="513*"/>
            </Grid.ColumnDefinitions>

            <Label x:Name="label" Content="Product Management"
                Grid.Column="2" Grid.Row="0" FontSize="36" Grid.ColumnSpan="2"
                HorizontalAlignment="Center" Width="466"/>

            <Label x:Name="label1" Margin ="2,2,2,2" Content="Product ID"
                Grid.Column="0" Grid.Row="1" Grid.ColumnSpan="2"/>
            <TextBox x:Name="txtProductID" Margin ="4,4,4,4" Grid.Column="2"
                Grid.Row="1" Text="" TextWrapping="Wrap" Grid.ColumnSpan="2"
                IsEnabled="False" />
        </Grid>
    </Grid>
</Window>
```



```

        <Label x:Name="label2" Margin ="2,2,2,2" Grid.Column="0"
Grid.Row="2" Content="Product Name" Grid.ColumnSpan="2" />
        <TextBox x:Name="txtProductName" Margin ="4,4,4,4"
Grid.Column="2" Grid.Row="2" Text="" TextWrapping="Wrap" Grid.ColumnSpan="2"
/>

        <Label x:Name="label8" Margin ="2,2,2,2" Content="Price"
Grid.Column="0" Grid.Row="3" Grid.ColumnSpan="2"/>
        <TextBox x:Name="txtPrice" Margin ="4,4,4,4" Grid.Column="2"
Grid.Row="3" Text="" TextWrapping="Wrap" Grid.ColumnSpan="2" />

        <Label x:Name="label3" Margin ="2,2,2,2" Content="Units In
Stock" Grid.Column="0" Grid.Row="4" Grid.ColumnSpan="2"/>
        <TextBox x:Name="txtUnitsInStock" Margin ="4,4,4,4"
Grid.Column="2" Grid.Row="4" Text="" TextWrapping="Wrap"
Grid.ColumnSpan="2" />

        <Label x:Name="label4" Margin ="2,2,2,2" Content="Category"
Grid.Column="0" Grid.Row="5" Grid.ColumnSpan="2"/>
        <ComboBox x:Name="cboCategory" Margin ="4,4,4,4" Grid.Column="2"
Grid.Row="5" Grid.ColumnSpan="2" />

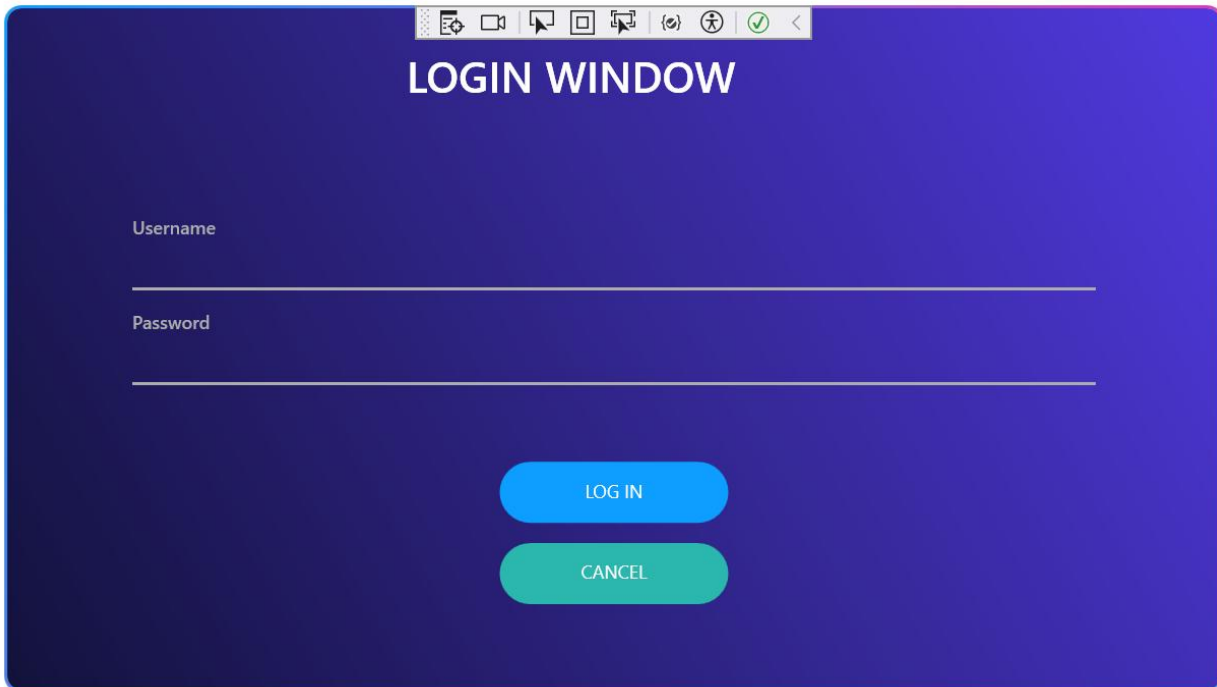
        <DataGrid x:Name="dgData" Margin ="4,4,4,63" Grid.Column="2"
Grid.Row="6" Grid.ColumnSpan="2" SelectionChanged="dgData_SelectionChanged"
/>

        <Button x:Name="btnCreate" Content="Create"
HorizontalAlignment="Left" Margin="29,365,0,16" Grid.Row="6"
Grid.RowSpan="2" Width="121" Background="#FF0099FF" BorderBrush="White"
Foreground="White" Grid.ColumnSpan="3" Grid.Column="1"
Click="btnCreate_Click"/>
        <Button x:Name="btnUpdate" Content="Update" Grid.Column="3"
HorizontalAlignment="Left" Margin="87,365,0,16" Grid.Row="6" Width="118"
Background="#FF1B7140" Foreground="White" Grid.RowSpan="2"
RenderTransformOrigin="0.37,0.2" Click="btnUpdate_Click"/>
        <Button x:Name="btnDelete" Content="Delete" Grid.Column="3"
HorizontalAlignment="Left" Margin="221,365,0,16" Grid.Row="6"
Grid.RowSpan="2" Width="127" Foreground="White" Background="#FFA2AA3D"
Click="btnDelete_Click"/>

        <Button x:Name="btnClose" Content="Close" Grid.Column="3"
HorizontalAlignment="Left" Margin="371,365,0,18" Grid.Row="6"
Grid.RowSpan="2" Width="120" Background="#FFEF8F18" Foreground="White"
Click="btnClose_Click"/>

    </Grid>
</Grid>
</Window>

```



```
<Window x:Class="WPFApp.LoginWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        xmlns:local="clr-namespace:WPFApp"
        mc:Ignorable="d"
        Title="LoginWindow" Height="450" Width="800"
        WindowStartupLocation="CenterScreen"
        WindowStyle="None"
        Background="Transparent"
        AllowsTransparency="True">
    <Grid>
        <Border CornerRadius="10"
            BorderThickness="2"
            Opacity="0.95">
            <Border.BorderBrush>
                <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                    <GradientStop Color="#0099FF" Offset="0"/>
                    <GradientStop Color="#DA34AE" Offset="0.75"/>
                    <GradientStop Color="#FF60B1E7" Offset="1"/>
                    <GradientStop Color="#FF596FDD" Offset="0.309"/>
                    <GradientStop Color="#FF8C57CA" Offset="0.484"/>
                </LinearGradientBrush>
            </Border.BorderBrush>
            <Border.Background>
                <LinearGradientBrush StartPoint="0,1" EndPoint="1,0">
                    <GradientStop Color="#060531" Offset="0"/>
                    <GradientStop Color="#FF472FDE" Offset="1"/>
                </LinearGradientBrush>
            </Border.Background>
        </Border>
    </Grid>
</Window>
```

```

        </LinearGradientBrush>
    </Border.Background>

    <Grid>
        <StackPanel Orientation="Horizontal"
HorizontalAlignment="Center"
Height="82" VerticalAlignment="Top" Width="632">
            <TextBlock Text="LOGIN WINDOW"
Foreground="White"
FontSize="28"
FontWeight="Medium"
FontFamily="Montserrat"
Cursor="Hand"
Margin="180,30,0,0" Width="377"
/>
        </StackPanel>
        <StackPanel
Orientation="Vertical"
Margin="82,102,82,68">
            <TextBlock Text="Username"
Foreground="DarkGray"
FontSize="12"
FontWeight="Medium"
FontFamily="Montserrat"
Margin="0,35,0,0"/>
            <TextBox x:Name="txtUser"
FontSize="13"
FontWeight="Medium"
FontFamily="Montserrat"
Foreground="White"
CaretBrush="LightGray"
BorderBrush="DarkGray"
BorderThickness="0,0,0,2"
Height="28"
VerticalContentAlignment="Center"
Margin="0,5,0,0"
>
            <TextBox.Background>
                <LinearGradientBrush></LinearGradientBrush>
            </TextBox.Background>
        </TextBox>

        <TextBlock Text="Password"
Foreground="DarkGray"
FontSize="12"
FontWeight="Medium"
FontFamily="Montserrat"
Margin="0,15,0,0"/>
        <PasswordBox x:Name="txtPass"
FontSize="13"
FontWeight="Medium"
FontFamily="Montserrat"

```

```

        Foreground="White"
        CaretBrush="LightGray"
        BorderBrush="DarkGray"
        BorderThickness="0,0,0,2"
        Height="28"
        VerticalContentAlignment="Center"
        Margin="0,5,0,0">
        <PasswordBox.Background>
            <LinearGradientBrush></LinearGradientBrush>
        </PasswordBox.Background>
    </PasswordBox>
    <Button x:Name="btnLogin"
        BorderThickness="0"
        Content="LOG IN"
        Foreground="White"
        FontSize="12"
        FontFamily="Montserrat"
        Cursor="Hand"
        Margin="0,50,0,0"
        Click="btnLogin_Click">
        <Button.Style>
            <Style TargetType="Button">
                <Setter Property="Background"
Value="#0099FF"/>
                <Style.Triggers>
                    <Trigger Property="IsMouseOver"
Value="True">
                        <Setter Property="Background"
Value="#28AEED"/>
                    </Trigger>
                </Style.Triggers>
            </Style>
        </Button.Style>
        <Button.Template>
            <ControlTemplate TargetType="Button">
                <Border Width="150" Height="40"
                    CornerRadius="20"
                    Background="{TemplateBinding
Background}">
                    <ContentPresenter
VerticalAlignment="Center"
HorizontalAlignment="Center"/>
                </Border>
            </ControlTemplate>
        </Button.Template>

    </Button>

</StackPanel>

```

```

        <StackPanel>
            <Button x:Name="btnCancel"
BorderThickness="0"
Content="CANCEL"
Foreground="White"
FontSize="12"
FontFamily="Montserrat"
Cursor="Hand"
Margin="20,350,20,0"
Click="btnCancel_Click">
                <Button.Style>
                    <Style TargetType="Button">
                        <Setter Property="Background"
Value="LightSeaGreen"/>
                        <Style.Triggers>
                            <Trigger Property="IsMouseOver"
Value="True">
                                <Setter Property="Background"
Value="SeaGreen"/>
                        </Trigger>
                    </Style.Triggers>
                </Style>
            </Button.Style>
            <Button.Template>
                <ControlTemplate TargetType="Button">
                    <Border Width="150" Height="40"
CornerRadius="20"
Background="{TemplateBinding Background}">
                        <ContentPresenter
VerticalAlignment="Center"
HorizontalAlignment="Center"/>
                    </Border>
                </ControlTemplate>
            </Button.Template>
        </Button>
    </StackPanel>
</Grid>
</Border>
</Grid>
</Window>

```

Step 03. Add a reference to the WPF project to Services Project

Step 04. Write codes for LoginWindow.xaml.cs:

```

1  using BusinessObjects;
2  using Services;
3  using System.Windows;
4
5  namespace WPFApp
6  {
7      /// <summary> Interaction logic for LoginWindow.xaml
8      2 references
9
10     public partial class LoginWindow : Window
11     {
12         private readonly IAccountService iAccountService;
13         0 references
14         public LoginWindow()
15         {
16             InitializeComponent();
17             iAccountService = new AccountService();
18         }
19
20         1 reference
21         private void btnLogin_Click(object sender, RoutedEventArgs e)
22         {
23
24         1 reference
25         private void btnCancel_Click(object sender, RoutedEventArgs e)
26         {
27             this.Close();
28         }
29     }
30 }

```

The details for btnLogin_Click() function:

```

19     private void btnLogin_Click(object sender, RoutedEventArgs e)
20     {
21         AccountMember account = iAccountService.GetAccountById(txtUser.Text);
22         if (account != null && account.MemberPassword.Equals(txtPass.Password)
23             && account.MemberRole == 1)
24         {
25             this.Hide();
26             MainWindow mainWindow = new MainWindow();
27             mainWindow.Show();
28         }
29         else
30         {
31             MessageBox.Show("You are not permission !");
32         }
33     }
34 }

```

Step 05. Write codes for MainWindow.xaml.cs:

```

1  using System;
2  using System.Windows;
3  using System.Windows.Controls;
4  using BusinessObjects;
5  using Services;
6
7  namespace WPFApp
8  {
9      /// <summary> Interaction logic for MainWindow.xaml
10     5 references
11
12     public partial class MainWindow : Window
13     {
14         private readonly IProductService iProductService;
15         private readonly ICategoryService iCategoryService;
16         1 reference
17         public MainWindow()...
18         1 reference
19
20         public void LoadCategoryList()...
21         4 references
22
23         public void LoadProductList()...
24         1 reference
25
26         private void Window_Loaded(object sender, RoutedEventArgs e)...
27         1 reference
28
29         private void btnCreate_Click(object sender, RoutedEventArgs e)...
30         1 reference
31
32         private void dgData_SelectionChanged(object sender, SelectionChangedEventArgs e)...
33         1 reference
34
35         private void btnClose_Click(object sender, RoutedEventArgs e)...
36         1 reference
37
38         private void btnUpdate_Click(object sender, RoutedEventArgs e)...
39         1 reference
40
41         private void btnDelete_Click(object sender, RoutedEventArgs e)...
42         1 reference
43
44         private void resetInput()...
45
46     }
47 }

```

The functions in details:

```

16  public MainWindow()
17  {
18      InitializeComponent();
19      iProductService = new ProductService();
20      iCategoryService = new CategoryService();
21  }
22  1 reference
23  public void LoadCategoryList()
24  {
25      try
26      {
27          var catList = iCategoryService.GetCategories();
28          cboCategory.ItemsSource = catList;
29          cboCategory.DisplayMemberPath = "CategoryName";
30          cboCategory.SelectedValuePath = "CategoryId";
31      }
32      catch (Exception ex)
33      {
34          MessageBox.Show(ex.Message, "Error on load list of categories");
35      }
36  }

37  public void LoadProductList()
38  {
39      try
40      {
41          var productList = iProductService.GetProducts();
42          dgData.ItemsSource = productList;
43      }
44      catch (Exception ex)
45      {
46          // MessageBox.Show(ex.Message, "Error on load list of products");
47      }
48      finally
49      {
50          resetInput();
51      }
52  }
53  1 reference
54  private void Window_Loaded(object sender, RoutedEventArgs e)
55  {
56      LoadCategoryList();
57      LoadProductList();

```

```

58 private void btnCreate_Click(object sender, RoutedEventArgs e)
59 {
60     try
61     {
62         Product product = new Product();
63         product.ProductName = txtProductName.Text;
64         product.UnitPrice = Decimal.Parse(txtPrice.Text);
65         product.UnitsInStock = short.Parse(txtUnitsInStock.Text);
66         product.CategoryId = Int32.Parse(cboCategory.SelectedValue.ToString());
67         iProductService.SaveProduct(product);
68     }
69     catch (Exception ex)
70     {
71         MessageBox.Show(ex.Message);
72     }
73     finally
74     {
75         LoadProductList();
76     }
77 }

78 private void dgData_SelectionChanged(object sender, SelectionChangedEventArgs e)
79 {
80     DataGrid dataGrid = sender as DataGrid;
81     DataGridRow row =
82         (DataGridRow)dataGrid.ItemContainerGenerator
83         .ContainerFromIndex(dataGrid.SelectedIndex);
84     DataGridCell RowColumn =
85         dataGrid.Columns[0].GetCellContent(row).Parent as DataGridCell;
86     string id = ((TextBlock)RowColumn.Content).Text;
87     Product product = iProductService.GetProductById(Int32.Parse(id));
88     txtProductID.Text = product.ProductId.ToString();
89     txtProductName.Text = product.ProductName;
90     txtPrice.Text = product.UnitPrice.ToString();
91     txtUnitsInStock.Text = product.UnitsInStock.ToString();
92     cboCategory.SelectedValue = product.CategoryId;
93 }
94 1 reference
95 private void btnClose_Click(object sender, RoutedEventArgs e)
96 {
97     this.Close();
98 }

```



```

98     private void btnUpdate_Click(object sender, RoutedEventArgs e)
99     {
100         try
101         {
102             if (txtProductID.Text.Length > 0)
103             {
104                 Product product = new Product();
105                 product.ProductId = Int32.Parse(txtProductID.Text);
106                 product.ProductName = txtProductName.Text;
107                 product.UnitPrice = Decimal.Parse(txtPrice.Text);
108                 product.UnitsInStock = short.Parse(txtUnitsInStock.Text);
109                 product.CategoryId = Int32.Parse(cboCategory.SelectedValue.ToString());
110                 iProductService.UpdateProduct(product);
111             }
112         }
113         else
114         {
115             MessageBox.Show("You must select a Product !");
116         }
117     }
118     catch (Exception ex)
119     {
120         MessageBox.Show(ex.Message);
121     }
122     finally
123     {
124         LoadProductList();
125     }
126 }

127     private void btnDelete_Click(object sender, RoutedEventArgs e)
128     {
129         try
130         {
131             if (txtProductID.Text.Length > 0)
132             {
133                 Product product = new Product();
134                 product.ProductId = Int32.Parse(txtProductID.Text);
135                 product.ProductName = txtProductName.Text;
136                 product.UnitPrice = Decimal.Parse(txtPrice.Text);
137                 product.UnitsInStock = short.Parse(txtUnitsInStock.Text);
138                 product.CategoryId = Int32.Parse(cboCategory.SelectedValue.ToString());
139                 iProductService.DeleteProduct(product);
140             }
141         }
142         else
143         {
144             MessageBox.Show("You must select a Product !");
145         }
146     }
147     catch (Exception ex)
148     {
149     }
150     finally
151     {
152         LoadProductList();
153     }
154 }
155
156

```

```
157 private void resetInput()  
158 {  
159     txtProductID.Text = "";  
160     txtProductName.Text = "";  
161     txtPrice.Text = "";  
162     txtUnitsInStock.Text = "";  
163     cboCategory.SelectedValue = 0;  
164 }  
165
```


Step 06. Open App.xaml and then update XAML code as follows:

```
<Application x:Class="WPFApp.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:WPFApp"
StartupUri="LoginWindow.xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```

Activity 05: Run the WPFApp project and test all actions